

Data Engineering/Data Analytics

Day-5

```
# Install required packages if needed
# Run this cell once
!pip install sqlalchemy pymysql mysql-connector-python
```

Installing Required Python Packages

```
# Install required packages if needed
# Run this cell once
!pip install sqlalchemy pymysql mysql-connector-python
```



What each package does

Package	Purpose
sqlalchemy	A powerful SQL toolkit and ORM. Helps create a database connection (engine) and work with databases easily.
pymysql	A pure Python MySQL driver. Used by SQLAlchemy to connect to MySQL.
mysql-connector-python	Official MySQL driver from Oracle. Another way to connect to MySQL directly.



After installation

You can now use these packages in your Python code, for example:

```
from sqlalchemy import create_engine
import pandas as pd
import pymysql
```

Run this cell only once
(unless you need to reinstall).

1. Import Libraries and Create MySQL Connection

```
] : import mysql.connector
import pandas as pd
from sqlalchemy import create_engine

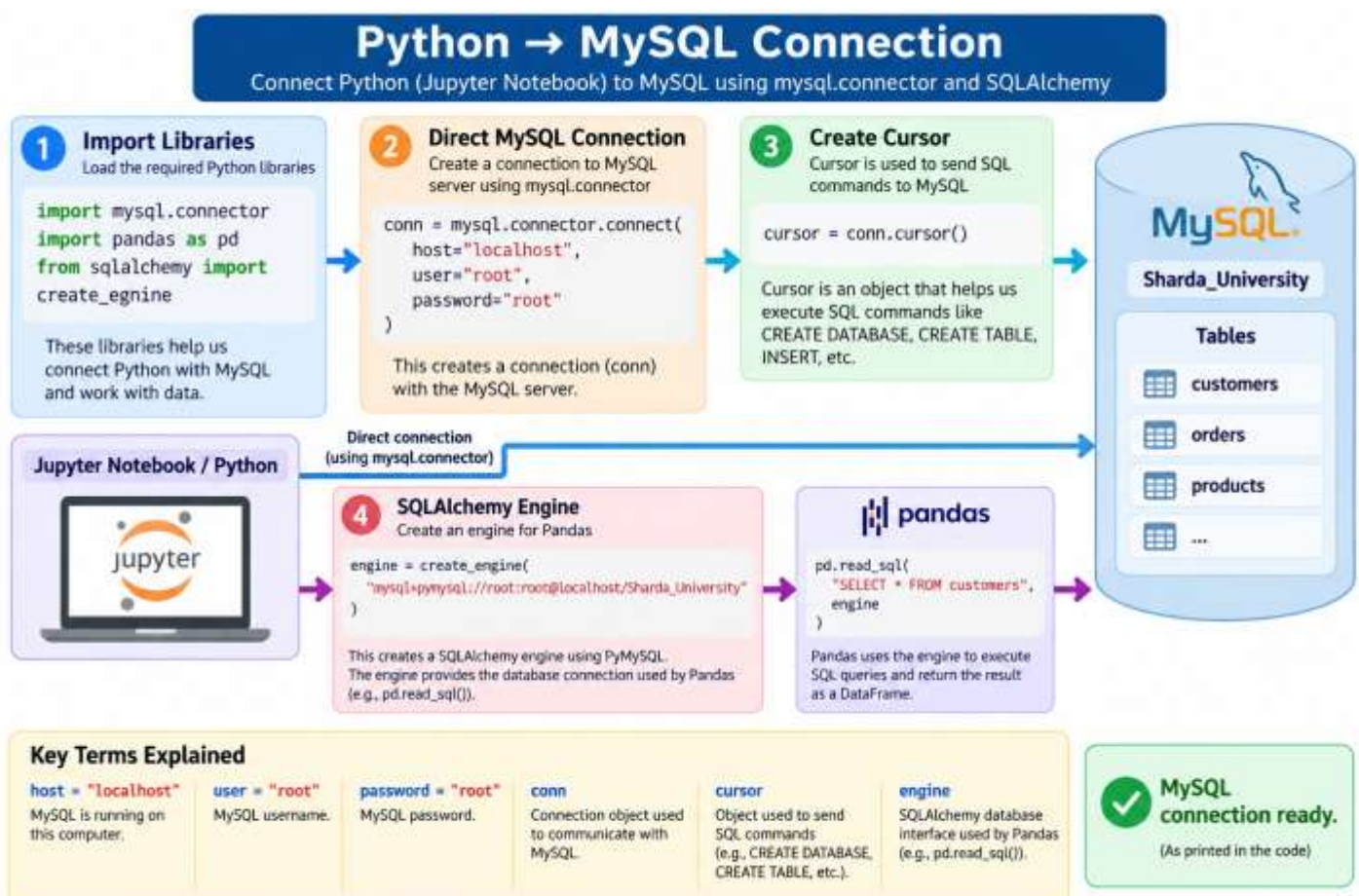
# Connection for database/table creation
conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="root"
)

cursor = conn.cursor()

# SQLAlchemy engine for Pandas
engine = create_engine(
    "mysql+pymysql://root:root@localhost/Sharda_University"
)

print("MySQL connection ready.")
```

MySQL connection ready.



This cell does **two types of database setup**:

1. Creates a direct MySQL connection using `mysql.connector`.
2. Creates a SQLAlchemy engine that Pandas can use to run SQL queries.

Object	Main use
<code>conn</code>	Direct MySQL operations
<code>cursor</code>	Execute SQL commands such as CREATE, INSERT, etc.
<code>engine</code>	Used by Pandas/SQLAlchemy to run queries
<code>pd.read_sql()</code>	Reads SQL results into a Pandas DataFrame

Easy way to remember

`cursor` = **send commands**

`engine` = **connect Pandas to database**

`conn` = **direct MySQL connection**

`pd.read_sql()` = **bring SQL result into Pandas**

2. Create Practice Database and Tables

```
# When Python connects to MySQL, we need an object that can send SQL commands to MySQL.
# That object is called a cursor.
# cursor is used to execute SQL statements and retrieve results from the database.
# execute() is a method of the cursor.
# Python tells the cursor to send USE Sharda_University to MySQL. MySQL then executes that command
```

```
cursor.execute("CREATE DATABASE IF NOT EXISTS Sharda_University")
cursor.execute("USE Sharda_University")
```

```
cursor.execute("DROP TABLE IF EXISTS orders")
cursor.execute("DROP TABLE IF EXISTS customers")
cursor.execute("DROP TABLE IF EXISTS products")
```

```
''' in Python means triple single quotes.
#It is mainly used to create a multi-line string.
```

```
cursor.execute('''
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    city VARCHAR(50)
)
```

```
cursor.execute('''
CREATE TABLE products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(50),
```

Creating Practice Database and Tables

Visualizing the structure and relationships (Primary & Foreign Keys)

1 Create Database and Use It

```
cursor.execute("CREATE DATABASE IF NOT EXISTS
Sharda_University")
cursor.execute("USE Sharda_University")
```

This creates a database named Sharda_University (if it doesn't exist) and makes it the active database.

2 Drop Tables (if they exist)

```
cursor.execute("DROP TABLE IF EXISTS orders")
cursor.execute("DROP TABLE IF EXISTS customers")
cursor.execute("DROP TABLE IF EXISTS products")
```

These commands remove the tables if they already exist, so we can create them again.

3 Create Tables

```
cursor.execute('''
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    city VARCHAR(50)
)

cursor.execute('''
CREATE TABLE products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(50),
    category VARCHAR(50),
    price DECIMAL(10,2)
)

cursor.execute('''
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    product_id INT,
    order_date DATE,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
)
```



- We create a database named Sharda_University.
- Inside this database, we create three tables: customers, products and orders.
- orders table has foreign keys that reference customers and products.



Relationship Diagram



Primary Keys uniquely identify each record.
Foreign Keys create a link between tables to maintain referential integrity.

3. SELECT, WHERE, DISTINCT, ORDER BY and LIMIT

```
#Pandas, run this SQL query using this MySQL database connection (engine) and give the result as a DataFrame
#Send this SQL query to MySQL through the SQLAlchemy engine and return the result in Pandas.
# SELECT
pd.read_sql("SELECT * FROM customers", engine)
```

	customer_id	name	city
0	1	Amit	Delhi
1	2	Neha	Mumbai
2	3	Ravi	Delhi
3	4	Priya	Pune
4	5	John	Delhi

3. SELECT, WHERE, DISTINCT, ORDER BY and LIMIT

Using the customers table

Table: customers

customer_id	name	city
1	Amit	Delhi
2	Neha	Mumbai
3	Ravi	Delhi
4	Priya	Pune
5	John	Delhi

This is our original data in the customers table.

1 SELECT

Retrieves columns from a table.

```
SELECT * FROM customers;
```

customer_id	name	city
1	Amit	Delhi
2	Neha	Mumbai
3	Ravi	Delhi
4	Priya	Pune
5	John	Delhi

Returns all rows and all columns from the customers table.

2 WHERE

Filters rows based on a condition.

```
SELECT * FROM customers
WHERE city = 'Delhi';
```

customer_id	name	city
1	Amit	Delhi
3	Ravi	Delhi
5	John	Delhi

Returns only customers who are from Delhi.

3 DISTINCT

Returns unique (different) values.

```
SELECT DISTINCT city FROM customers;
```

city
Delhi
Mumbai
Pune

Returns unique city names.
Duplicates are removed.

4 ORDER BY

Sorts the result in ascending or descending order.

```
SELECT * FROM customers
ORDER BY name ASC;
```

customer_id	name	city
1	Amit	Delhi
5	John	Delhi
2	Neha	Mumbai
4	Priya	Pune
3	Ravi	Delhi

Returns all customers sorted by name (A to Z).

5 LIMIT

Returns only a specified number of rows.

```
SELECT * FROM customers
LIMIT 3;
```

customer_id	name	city
1	Amit	Delhi
2	Neha	Mumbai
3	Ravi	Delhi

Returns only the first 3 rows.



You can also combine them!

```
SELECT DISTINCT city FROM customers ORDER BY city LIMIT 2;
```

Returns the first 2 unique cities in alphabetical order.

4. AND, OR, IN, BETWEEN and LIKE

```
pd.read_sql("""
SELECT * FROM products
WHERE price BETWEEN 1000 AND 10000
""", engine)
```

	product_id	product_name	category	price
0	2	Mouse	Electronics	1000.0
1	3	Chair	Furniture	5000.0
2	4	Desk	Furniture	10000.0

4. AND, OR, IN, BETWEEN and LIKE

Filter data using different conditions

Table: products (All Data)

product_id	product_name	category	price
1	Laptop	Electronics	60000
2	Mouse	Electronics	1000
3	Chair	Furniture	5000
4	Desk	Furniture	10000
5	Keyboard	Electronics	1500
6	Monitor	Electronics	20000

This is the complete products table.

1 BETWEEN

Select values within a range (inclusive).

```
SELECT * FROM products
WHERE price BETWEEN 1000 AND 10000;
```

product_id	product_name	category	price
2	Mouse	Electronics	1000
3	Chair	Furniture	5000
4	Desk	Furniture	10000

Returns products with price between 1000 and 10000 (including both).

2 AND

All conditions must be true.

```
SELECT * FROM products
WHERE category = 'Electronics'
AND price > 1000;
```

product_id	product_name	category	price
5	Keyboard	Electronics	1500
6	Monitor	Electronics	20000
1	Laptop	Electronics	60000

Returns products that are Electronics and have price greater than 1000.

3 OR

At least one condition must be true.

```
SELECT * FROM products
WHERE category = 'Furniture'
OR price < 1500;
```

product_id	product_name	category	price
2	Mouse	Electronics	1000
3	Chair	Furniture	5000
4	Desk	Furniture	10000

Returns products that are Furniture or have price less than 1500.

4 IN

Match any value in a list.

```
SELECT * FROM products
WHERE category IN ('Electronics', 'Furniture');
```

product_id	product_name	category	price
1	Laptop	Electronics	60000
2	Mouse	Electronics	1000
3	Chair	Furniture	5000
4	Desk	Furniture	10000
5	Keyboard	Electronics	1500
6	Monitor	Electronics	20000

Returns products where category is either Electronics or Furniture (same as all rows in this case).

5 LIKE

Search for text pattern (wildcards).

```
SELECT * FROM products
WHERE product_name LIKE 'D%';
```

product_id	product_name	category	price
4	Desk	Furniture	10000

Returns products whose name starts with 'D' (D followed by any characters).

Common LIKE patterns:

- 'D%' → starts with D
- '%top' → ends with top
- '%ouse%' → contains 'ouse'

These operators help you filter data based on different conditions in SQL.

5. Aggregate Functions

```
[42]: pd.read_sql("""
SELECT
    COUNT(*) AS total_products,
    AVG(price) AS average_price,
    MIN(price) AS minimum_price,
    MAX(price) AS maximum_price,
    SUM(price) AS total_price
FROM products
""", engine)
```

```
[42]:
```

	total_products	average_price	minimum_price	maximum_price	total_price
0	4	19000.0	1000.0	60000.0	76000.0

5. Aggregate Functions in SQL

Get summary statistics from your data

Products Table (Input Data)

product_id	product_name	category	price
1	Laptop	Electronics	60000
2	Mouse	Electronics	1000
3	Chair	Furniture	5000
4	Desk	Furniture	10000

This table has 4 products with different prices.

SQL Query

```
SELECT
COUNT(*) AS total_products,
AVG(price) AS average_price,
MIN(price) AS minimum_price,
MAX(price) AS maximum_price,
SUM(price) AS total_price
FROM products;
```

Result (Output)

total_products	average_price	minimum_price	maximum_price	total_price
4	19000.0	1000.0	60000.0	76000.0

The query gives a single row with summary information about the products table.

What each function does

Function	Purpose	Example (on products table)	Result	Visual Explanation
COUNT(*)	Counts total number of rows	SELECT COUNT(*) FROM products;	4	 Counts all products
AVG(price)	Finds the average value of a column	SELECT AVG(price) FROM products;	19000.0	$(60000 + 1000 + 5000 + 10000) / 4 = 19000$ Sum of all prices divided by number of products
MIN(price)	Finds the smallest value	SELECT MIN(price) FROM products;	1000.0	1000 Smallest price (Mouse)
MAX(price)	Finds the largest value	SELECT MAX(price) FROM products;	60000.0	60000 Largest price (Laptop)
SUM(price)	Finds the total sum	SELECT SUM(price) FROM products;	76000.0	$60000 + 1000 + 5000 + 10000 = 76000$ Total value of all products



Key Takeaway

Aggregate functions perform calculations on multiple rows and return a single value (or a single row). They are very useful for getting summary information from your data.

WHERE vs HAVING

- `WHERE` filters rows before grouping.
- `HAVING` filters groups after `GROUP BY`.

```
[43]: pd.read_sql("""
SELECT category, AVG(price) AS avg_price
FROM products
GROUP BY category
HAVING AVG(price) > 5000
""", engine)
```

```
[43]:
```

	category	avg_price
0	Electronics	30500.0
1	Furniture	7500.0

WHERE vs HAVING

Both are used to filter data, but they work at different stages in the query.

1. Original Data (products table)

product_id	product_name	category	price
1	Laptop	Electronics	60000
2	Mouse	Electronics	1000
3	Chair	Furniture	5000
4	Desk	Furniture	10000
5	Keyboard	Electronics	1500
6	Monitor	Electronics	20000

This is the complete data in the products table.

2 WHERE (Row Filtering)

- Filters individual rows **before** grouping.
- Works on column values.
- Does not use aggregate functions (e.g., AVG, SUM).

Syntax:

```
SELECT columns
FROM table
WHERE condition;
```

3 HAVING (Group Filtering)

- Filters groups **after** GROUP BY.
- Works on aggregate functions.
- Used with GROUP BY.

Syntax:

```
SELECT columns, AGG_FUNCTION()
FROM table
GROUP BY columns
HAVING condition;
```

Example 1: Using WHERE

Get products with price > 5000 (row-level filter)

```
SELECT * FROM products
WHERE price > 5000;
```

product_id	product_name	category	price
1	Laptop	Electronics	60000
4	Desk	Furniture	10000
6	Monitor	Electronics	20000

WHERE filters rows before grouping.

Example 2: Using HAVING

Get categories with average price > 5000

```
SELECT category, AVG(price) AS avg_price
FROM products
GROUP BY category
HAVING AVG(price) > 5000;
```

category	avg_price
Electronics	30500.0
Furniture	7500.0

HAVING filters groups after GROUP BY using an aggregate function (AVG).

Step-by-Step Process (for HAVING example)



Key Differences

Aspect	WHERE	HAVING
When it is applied	Before grouping	After grouping
What it filters	Individual rows	Grouped rows
Can use aggregate functions	No	Yes (e.g., AVG, SUM, COUNT)
Used with GROUP BY	Not required	Required
Example condition	price > 5000	AVG(price) > 5000



Key Takeaway

- Use **WHERE** to filter rows based on a condition on column values.
- Use **HAVING** to filter groups based on a condition on aggregate values.
- WHERE comes before GROUP BY, and HAVING comes after GROUP BY in the logical order of SQL execution.

WHERE filters data, HAVING filters summaries.

6. CASE WHEN

```
4]: pd.read_sql("""
SELECT product_name, price,
       CASE
         WHEN price >= 10000 THEN 'Expensive'
         WHEN price >= 5000 THEN 'Medium'
         ELSE 'Low'
       END AS price_level
FROM products
""", engine)
```

```
4]:
```

	product_name	price	price_level
0	Laptop	60000.0	Expensive
1	Mouse	1000.0	Low
2	Chair	5000.0	Medium
3	Desk	10000.0	Expensive

6. CASE WHEN in SQL

Create new values/labels based on conditions

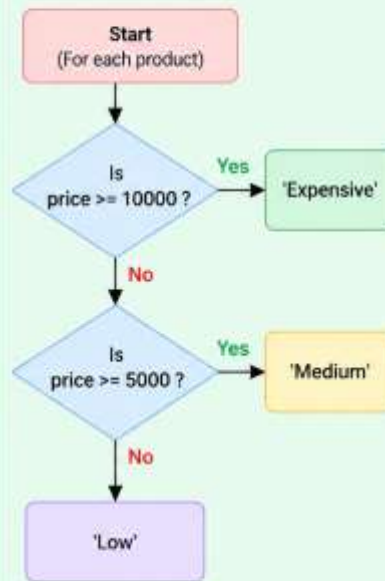
SQL Query

```
SELECT product_name, price,  
CASE  
  WHEN price >= 10000 THEN 'Expensive'  
  WHEN price >= 5000 THEN 'Medium'  
  ELSE 'Low'  
END AS price_level  
FROM products;
```

How it works?

- 1 For each row, SQL checks the price value.
- 2 If price ≥ 10000 , label as 'Expensive'.
- 3 Else if price ≥ 5000 , label as 'Medium'.
- 4 Otherwise, label as 'Low'.
- 5 The CASE expression creates a new column `price_level` based on these conditions.

Decision Logic (Flowchart)



Input Table: products

product_id	product_name	category	price
1	Laptop	Electronics	60000
2	Mouse	Electronics	1000
3	Chair	Furniture	5000
4	Desk	Furniture	10000

Query Result: with price_level

product_name	price	price_level
Laptop	60000.0	Expensive
Mouse	1000.0	Low
Chair	5000.0	Medium
Desk	10000.0	Expensive



Key Takeaway:

CASE WHEN helps you create meaningful categories or labels based on conditions. It is very useful for data analysis and reporting.



You can use any number of WHEN conditions!

7. String and Date Functions

```
[45]: pd.read_sql("""  
SELECT name,  
        UPPER(name) AS upper_name,  
        LENGTH(name) AS name_length  
FROM customers  
""", engine)
```

```
[45]:
```

	name	upper_name	name_length
0	Amit	AMIT	4
1	Neha	NEHA	4
2	Ravi	RAVI	4
3	Priya	PRIYA	5
4	John	JOHN	4

```
[46]: pd.read_sql("""
SELECT order_id, order_date,
       YEAR(order_date) AS order_year,
       MONTH(order_date) AS order_month
FROM orders
""", engine)
```

```
[46]:
```

	order_id	order_date	order_year	order_month
0	101	2026-09-01	2026	9
1	102	2026-09-02	2026	9
2	103	2026-09-02	2026	9
3	104	2026-09-03	2026	9
4	105	2026-09-04	2026	9

7. String and Date Functions in SQL

Work with text (strings) and date values in your data

A. String Functions Used to manipulate text (string) values.

1. Original Data (customers table)

name
Amit
Neha
Ravi
Priya
John

2 SQL Query

```
SELECT name,
       UPPER(name) AS upper_name,
       LENGTH(name) AS name_length
FROM customers;
```

- **UPPER(name)** → converts text to uppercase.
- **LENGTH(name)** → returns the number of characters in the text.

3 Result

name	upper_name	name_length
Amit	AMIT	4
Neha	NEHA	4
Ravi	RAVI	4
Priya	PRIYA	5
John	JOHN	4

For each customer, the name is converted to uppercase and the length of the name is calculated.

B. Date Functions Used to work with date values (extract parts, format, calculate differences, etc.).

1. Original Data (orders table)

order_id	order_date
101	2026-09-01
102	2026-09-02
103	2026-09-02
104	2026-09-03
105	2026-09-04

2 SQL Query (common date functions)

```
SELECT order_id,
       order_date,
       YEAR(order_date) AS order_year,
       MONTH(order_date) AS order_month,
       DAY(order_date) AS order_day
FROM orders;
```

- **YEAR(date)** → extracts the year.
- **MONTH(date)** → extracts the month (1–12).
- **DAY(date)** → extracts the day (1–31).

3 Result

order_id	order_date	order_year	order_month	order_day
101	2026-09-01	2026	9	1
102	2026-09-02	2026	9	2
103	2026-09-02	2026	9	2
104	2026-09-03	2026	9	3
105	2026-09-04	2026	9	4

For each order, the year, month and day are extracted from the order_date.



Key Takeaway:

- String functions help you work with text data (e.g., UPPER, LOWER, LENGTH, SUBSTRING).
- Date functions help you extract and manipulate date parts (e.g., YEAR, MONTH, DAY).
- These functions are very useful for data cleaning, analysis and reporting.

8. INNER JOIN

```
[47]: pd.read_sql("""
SELECT c.name, o.order_id, o.order_date
FROM customers c
INNER JOIN orders o
      ON c.customer_id = o.customer_id
""", engine)
```

```
[47]:
```

	name	order_id	order_date
0	Amit	101	2026-09-01
1	Amit	102	2026-09-02
2	Neha	103	2026-09-02
3	Ravi	104	2026-09-03
4	Ravi	105	2026-09-04

8. INNER JOIN

Combines rows from two tables when there is a matching value in both tables

Table: customers (c)

customer_id	name
1	Amit
2	Neha
3	Ravi
4	Priya
5	John

List of customers

INNER JOIN

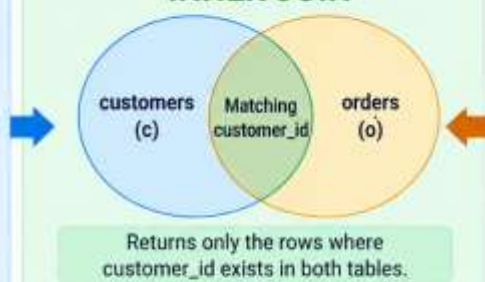


Table: orders (o)

order_id	customer_id	order_date
101	1	2026-09-01
102	1	2026-09-02
103	2	2026-09-02
104	3	2026-09-03
105	3	2026-09-04
106	6	2026-09-05

List of orders (Note: customer_id 6 does not exist in customers table)

SQL Query

```
SELECT c.name, o.order_id, o.order_date
FROM customers c
INNER JOIN orders o
ON c.customer_id = o.customer_id;
```

How it works?

- 1 Take each customer from customers table.
- 2 Find matching orders in orders table using `c.customer_id = o.customer_id`.
- 3 Return only the matching rows (from both tables).

Result

name	order_id	order_date
Amit	101	2026-09-01
Amit	102	2026-09-02
Neha	103	2026-09-02
Ravi	104	2026-09-03
Ravi	105	2026-09-04

Key Takeaways

- ✓ INNER JOIN returns only matching rows from both tables.
- ✓ Rows without a match (e.g., `customer_id = 6`) are not included in the result.
- ✓ Useful for combining related data from multiple tables.
- ✓ Commonly used in real-world databases (e.g., customers and orders).

INNER JOIN = Only matching data from both tables.

9. JOIN Three Tables

```
[48]: pd.read_sql("""
SELECT c.name, p.product_name, o.quantity
FROM orders o
JOIN customers c
      ON o.customer_id = c.customer_id
JOIN products p
      ON o.product_id = p.product_id
""", engine)
```

```
[48]:
```

	name	product_name	quantity
0	Amit	Laptop	1
1	Amit	Mouse	2
2	Ravi	Mouse	3
3	Neha	Chair	1
4	Ravi	Desk	1

9. JOIN Three Tables

Combine data from multiple related tables to get a complete view

1. customers (c)

Customer details

customer_id	name
1	Amit
2	Neha
3	Ravi
4	Priya
5	John

1 ∞
c.customer_id
=
o.customer_id

2. orders (o)

Which customer placed an order

order_id	customer_id	order_date
101	1	2026-09-01
102	1	2026-09-01
103	3	2026-09-02
104	2	2026-09-02
105	3	2026-09-03

∞ 1
o.product_id
=
p.product_id

3. products (p)

Product details

product_id	product_name
1	Laptop
2	Mouse
3	Chair
4	Desk

SQL Query (Join Three Tables)

```
SELECT c.name, p.product_name, o.quantity
FROM orders o
JOIN customers c
  ON o.customer_id = c.customer_id
JOIN products p
  ON o.product_id = p.product_id;
```

Result (Joined Data)

	name	product_name	quantity
0	Amit	Laptop	1
1	Amit	Mouse	2
2	Ravi	Mouse	3
3	Neha	Chair	1
4	Ravi	Desk	1

How it Works?

- 1 Join orders with customers using `o.customer_id = c.customer_id` (gets customer name for each order).
- 2 Then join the result with products using `o.product_id = p.product_id` (gets product name for each order).
- 3 Select the required columns (customer name, product name, quantity).

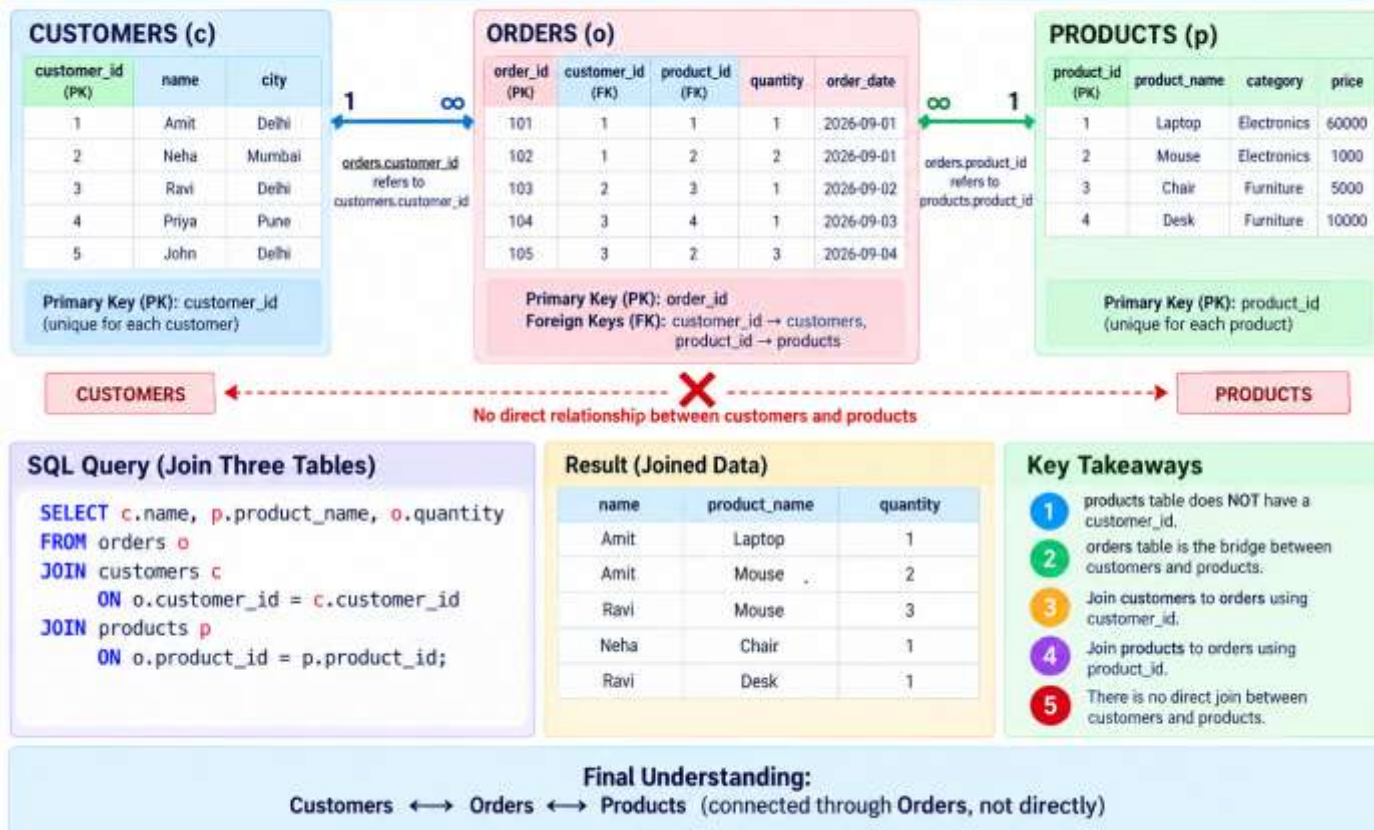
Key Takeaways

- ✓ You can join more than two tables in a single query.
- Use `JOIN ... ON` to specify how the tables are related.
- This helps to get a complete view by combining data from multiple tables.
- Very useful in real-world databases (e.g., customers, orders, products).

JOIN helps you combine related data from multiple tables to get meaningful information.

Join Three Tables: How the Tables are Connected

The orders table acts as a bridge between customers and products.



10. LEFT JOIN – Customers with No Orders ¶

```
9]: pd.read_sql("""
SELECT c.name
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL
""", engine)
```

```
9]:      name
0   Priya
1    John
```


10. LEFT JOIN – Customers with No Orders

Show all customers, including those who have not placed any orders

Customers (c)

customer_id	name	city
1	Amit	Delhi
2	Neha	Mumbai
3	Ravi	Delhi
4	Priya	Pune
5	John	Delhi

Primary Key: customer_id
(unique for each customer)

Orders (o)

order_id	customer_id	order_date
101	1	2026-09-01
102	1	2026-09-02
103	2	2026-09-02
104	3	2026-09-03

Foreign Key: customer_id
(refers to customers.customer_id)

Result (Customers with No Orders)

name
Priya
John

These customers do not have any matching orders.

SQL Query

```
SELECT c.name
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

How it Works?

- 1 LEFT JOIN returns all customers (even if they do not have orders).
- 2 For customers without orders, the columns from orders (o.*) will be NULL.
- 3 We filter with WHERE o.order_id IS NULL to get only those customers.



Key Takeaway: LEFT JOIN is useful to find records in the left table that do not have a match in the right table. In this case, we find customers who have not placed any orders.

11. UNION

```
0]: pd.read_sql("""
SELECT city FROM customers
UNION
SELECT category FROM products
""", engine)
```

```
0]:
```

	city
0	Delhi
1	Mumbai
2	Pune
3	Electronics
4	Furniture

11. UNION in SQL

Combines the result sets of two SELECT statements and removes duplicate rows.

CUSTOMERS

(Table: customers)

customer_id	city
1	Delhi
2	Mumbai
3	Delhi
4	Pune

SELECT city
FROM customers

SELECT category
FROM products

PRODUCTS

(Table: products)

product_id	category
1	Electronics
2	Electronics
3	Furniture
4	Furniture

UNION

Combines both result sets
and removes duplicate values

SQL Query

```
SELECT city  
FROM customers  
UNION  
SELECT category  
FROM products;
```

Result (Distinct Values)

	city
0	Delhi
1	Mumbai
2	Pune
3	Electronics
4	Furniture



Why Delhi appears once?

Delhi occurs twice in the customers table, but UNION removes duplicate values, so it appears only once in the final result.



Key Takeaway: UNION combines compatible SELECT results and removes duplicate values.

12. Subquery

```
pd.read_sql("""
SELECT product_name, price
FROM products
WHERE price > (SELECT AVG(price) FROM products)
""", engine)
```

	product_name	price
0	Laptop	60000.0

12. Subquery in SQL

Find products whose price is greater than the average price of all products

Products Table

product_id	product_name	price
1	Laptop	60000
2	Mouse	1000
3	Chair	5000
4	Desk	10000

This table contains all products with their prices.

Step 1: Subquery (Find Average Price)

```
SELECT AVG(price)
FROM products;
```

Average Price

19,000

((60000 + 1000 + 5000 + 10000) / 4)

Step 2: Outer Query (Filter Products)

```
SELECT product_name, price
FROM products
WHERE price > (SELECT AVG(price)
FROM products);
```

This is the subquery

How It Works?

- 1 The subquery first calculates the average price of all products (19,000).
- 2 The outer query then selects products with price greater than 19,000.
- 3 In our data, only Laptop (60,000) is greater than the average price.

Result

product_name	price
Laptop	60000.0

Only **Laptop** is returned because its price (60,000) is greater than the average price (19,000).



Key Takeaway: A **subquery** is a query inside another query. It can be used to calculate a value (e.g., average) and use it in the outer query for filtering.

13. Common Table Expression (CTE)

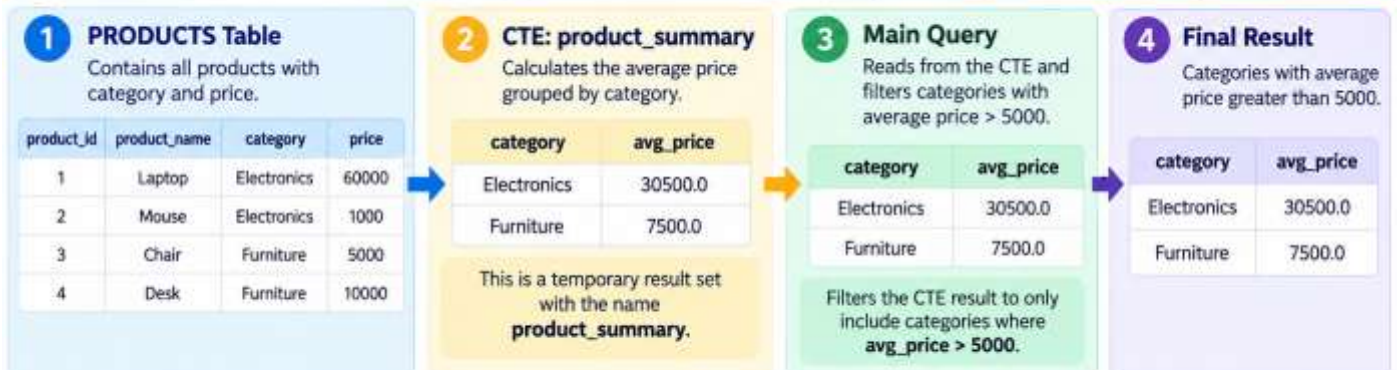
```
] : pd.read_sql("""
WITH product_summary AS (
    SELECT category, AVG(price) AS avg_price
    FROM products
    GROUP BY category
)
SELECT *
FROM product_summary
WHERE avg_price > 5000
""", engine)
```

```
] :
```

	category	avg_price
0	Electronics	30500.0
1	Furniture	7500.0

13. Common Table Expression (CTE)

A CTE creates a temporary named result that can be queried by the main query.



SQL Code (from the example)

```
WITH product_summary AS (
    SELECT category, AVG(price) AS avg_price
    FROM products
    GROUP BY category
)
SELECT *
FROM product_summary
WHERE avg_price > 5000;
```

How it works?

- 1 The CTE first calculates the average price for each category using **GROUP BY**.
- 2 The result is given a temporary name: **product_summary**.
- 3 The main query reads from this named result (**product_summary**).
- 4 The **WHERE** clause filters the resulting categories to include only those with **avg_price > 5000**.



Key Takeaway:

CTE = give a query result a name, then use that name in the main query.


```
[53]: # ROW_NUMBER
# ROW_NUMBER() gives a unique number to each row.
# Calculate the row number using the rules inside OVER().
# OVER() is what makes ROW_NUMBER() a window function.
# Divide the products into separate groups based on category.
# Within each category, arrange products from highest price to lowest price

pd.read_sql("""
SELECT product_name, category, price,
       ROW_NUMBER() OVER (
         PARTITION BY category
         ORDER BY price DESC
       ) AS row_num
FROM products
""", engine)
```

```
[53]:
```

	product_name	category	price	row_num
0	Laptop	Electronics	60000.0	1
1	Mouse	Electronics	1000.0	2
2	Desk	Furniture	10000.0	1
3	Chair	Furniture	5000.0	2

ROW_NUMBER() with PARTITION BY and ORDER BY

Divide products into categories and give a unique row number within each category (arranged by price from highest to lowest).

1 Products Table (Input)

product_id	product_name	category	price
1	Laptop	Electronics	60000
2	Mouse	Electronics	1000
3	Chair	Furniture	5000
4	Desk	Furniture	10000

This table contains all products with their category and price.

2 SQL Query

```
SELECT product_name, category, price,
       ROW_NUMBER() OVER (
         PARTITION BY category
         ORDER BY price DESC
       ) AS row_num
FROM products;
```

3 How it Works?

- PARTITION BY category**
Divides the data into separate groups (Electronics and Furniture).
- ORDER BY price DESC**
Arranges products within each category from highest price to lowest price.
- ROW_NUMBER()**
Assigns a unique number starting from 1 within each category based on the order.

4 Step-by-Step Result

Step 1: Split into Categories

Electronics Category	
product_name	price
Laptop	60000
Mouse	1000

Furniture Category	
product_name	price
Chair	5000
Desk	10000

Step 2: Order by Price (DESC) within each category

Electronics (ordered by price DESC)		
product_name	price	row_num
Laptop	60000	1
Mouse	1000	2

Furniture (ordered by price DESC)		
product_name	price	row_num
Desk	10000	1
Chair	5000	2

Step 3: Final Result (Combined)

product_name	category	price	row_num
Laptop	Electronics	60000.0	1
Mouse	Electronics	1000.0	2
Desk	Furniture	10000.0	1
Chair	Furniture	5000.0	2

 **Key Takeaway:** ROW_NUMBER() gives a unique number to each row within a group (category) based on the specified order.

```
[54]: # RANK
pd.read_sql("""
SELECT product_name, price,
       RANK() OVER (ORDER BY price DESC) AS price_rank
FROM products
""", engine)
```

```
[54]:
```

	product_name	price	price_rank
0	Laptop	60000.0	1
1	Desk	10000.0	2
2	Chair	5000.0	3
3	Mouse	1000.0	4

RANK() in SQL

Assigns a rank to each row based on the specified order.

1 Products Table (Input)

List of products with their prices.

product_id	product_name	price
1	Laptop	60000
2	Mouse	1000
3	Chair	5000
4	Desk	10000

This table contains all products with their prices.

2 SQL Query

Use **RANK()** to assign rank based on price (from highest to lowest).

```
SELECT product_name, price,
       RANK() OVER (ORDER BY price DESC)
       AS price_rank
FROM products;
```

ORDER BY price DESC means:
Highest price gets rank 1.

3 Result (Output)

Products ranked by price (highest to lowest).

product_name	price	price_rank
Laptop	60000.0	1
Desk	10000.0	2
Chair	5000.0	3
Mouse	1000.0	4

Highest Price
↑
↓
Lowest Price

4 How It Works (Step by Step)

- RANK()** looks at the **price** column for each row.
- ORDER BY price DESC** sorts the rows from highest to lowest price.
- The highest price (60000) gets **rank 1**.
- The next highest price (10000) gets **rank 2**, and so on.



Important Points for Beginners

- RANK()** assigns a rank to each row based on the order.
- ORDER BY price DESC** means highest price first.
- Ranks start from 1.
- If two products have the same price, **RANK()** will give them the same rank and skip the next rank (e.g., 1, 1, 3).
- If you want consecutive ranks without gaps, use **DENSE_RANK()**.



Key Takeaway:

RANK() helps you rank rows based on a column value, useful for finding top items, leaderboards, or sorted results.

```
[55]: # Running total
pd.read_sql("""
SELECT order_id, order_date, quantity,
       SUM(quantity) OVER (
         ORDER BY order_date, order_id
       ) AS running_quantity
FROM orders
""", engine)
```

```
[55]:
```

	order_id	order_date	quantity	running_quantity
0	101	2026-09-01	1	1.0
1	102	2026-09-02	2	3.0
2	103	2026-09-02	1	4.0
3	104	2026-09-03	1	5.0
4	105	2026-09-04	3	8.0

Running Total using SUM() OVER()

This is a Window Function in SQL

Yes, this is a
Window Function

1 Orders Table (Input)

order_id	order_date	quantity
101	2026-09-01	1
102	2026-09-02	2
103	2026-09-02	1
104	2026-09-03	1
105	2026-09-04	3

This table contains each order with the date and quantity.

2 SQL Query

```
SELECT order_id, order_date, quantity,
       SUM(quantity) OVER (
         ORDER BY order_date, order_id
       ) AS running_quantity
FROM orders;
```

3 Result (Output)

order_id	order_date	quantity	running_quantity
101	2026-09-01	1	1.0
102	2026-09-02	2	3.0
103	2026-09-02	1	4.0
104	2026-09-03	1	5.0
105	2026-09-04	3	8.0

running_quantity is the cumulative (running) total of quantity up to the current row.

4 How the Running Total is Calculated

Row	order_id	order_date	quantity	Calculation	running_quantity
1	101	2026-09-01	1	1	1
2	102	2026-09-02	2	1 + 2	3
3	103	2026-09-02	1	1 + 2 + 1	4
4	104	2026-09-03	1	1 + 2 + 1 + 1	5
5	105	2026-09-04	3	1 + 2 + 1 + 1 + 3	8

5 Key Points

- 1 SUM() OVER() is a window function.
- 2 ORDER BY order_date, order_id specifies the order in which the running total is calculated.
- 3 The window includes all rows from the start up to the current row (default frame).
- 4 The original rows are not grouped or collapsed – we still get one row per order.
- 5 Useful for running totals, cumulative sums, moving averages, rankings, etc.



Key Takeaway:

This is a **Window Function** because it uses SUM() with OVER().
Window functions perform calculations across a set of rows related to the current row.

```
[56]: # LAG
pd.read_sql("""
SELECT order_id, order_date, quantity,
       LAG(quantity) OVER (
         ORDER BY order_date, order_id
       ) AS previous_quantity
FROM orders
""", engine)
```

```
[56]:
```

	order_id	order_date	quantity	previous_quantity
0	101	2026-09-01	1	NaN
1	102	2026-09-02	2	1.0
2	103	2026-09-02	1	2.0
3	104	2026-09-03	1	1.0
4	105	2026-09-04	3	1.0

LAG() in SQL

LAG() gives the value from the previous row (based on the specified order).

1 Input Table: orders

order_id	order_date	quantity
101	2026-09-01	1
102	2026-09-02	2
103	2026-09-02	1
104	2026-09-03	1
105	2026-09-04	3

This table contains each order with the date and quantity.

2 SQL Query

```
SELECT order_id, order_date,
       LAG(quantity) OVER (
         ORDER BY order_date,
                  order_id
       ) AS previous_quantity
FROM orders;
```

3 How LAG() Works?

- 1 The rows are ordered by order_date, order_id.
- 2 For each row, LAG(quantity) looks at the quantity from the previous row.
- 3 The first row has no previous row, so it returns **NULL** (shown as NaN in pandas).
- 4 For all other rows, it shows the quantity from the row just above.

4 Step-by-Step Calculation

Row	order_id	order_date	quantity	Previous row quantity	previous_quantity (LAG)
1	101	2026-09-01	1	— (no previous row)	NULL (NaN)
2	102	2026-09-02	2	1	1
3	103	2026-09-02	1	2	2
4	104	2026-09-03	1	1	1
5	105	2026-09-04	3	1	1

First row has no previous value

Previous quantity = 1

Previous quantity = 2

Previous quantity = 1

Previous quantity = 1

5 Final Result (Output)

order_id	order_date	quantity	previous_quantity
101	2026-09-01	1	NaN
102	2026-09-02	2	1.0
103	2026-09-02	1	2.0
104	2026-09-03	1	1.0
105	2026-09-04	3	1.0



Key Takeaways

- LAG() is a window function.
- It gives the value from a previous row.
- The first row returns NULL because there is no previous row.
- Useful for comparing with previous values (e.g., day-over-day analysis).